



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025 26)
Faculty Name	NM Deepika
Student Name	KOTA MOHITH
Roll Number	2520030520
Branch	CSE
Course Outcome	CO-1

Case Study Topic:WhatsApp Delivered-Receipts Ordering — High-Throughput AVL Design Reality Check

Work Done Summary:

Analyzed a real-time messaging backend architecture tracking in-flight "message delivered" receipts.

Evaluated a proposed production design using a plain Binary Search Tree (BST) keyed by receipt-timestamp in descending order.

Modeled system behavior under a peak load of 5,000 messages/second sustained over one hour with strictly monotonic (always increasing) timestamps.

Determined the structural degradation (skewness) of a plain BST under sequential insertion inputs and calculated the resulting height.

Derived the worst-case time complexity in pointer hops for finding the oldest pending receipt via a leftmost descent path.

Proposed an AVL Tree alternative, comparing pointer hop counts and calculating processing latencies against a target Service Level Agreement (SLA) of 500 μ s.

Implementation Details:

Data Organization: Nodes are organized in a self-balancing binary search structure where the keys are timestamps sorted in descending order (newer timestamps route left, older timestamps route right).

Core Read Operation: The "find oldest pending receipt" query relies on a localized **leftmost descent** down the tree to isolate the oldest elements quickly.

The Performance Bottleneck: When data arrives sequentially (strictly monotonic timestamps), an unbalancing tree continuously chains elements along a single direction, transforming logarithmic tree operations into linear operations.

The Self-Balancing Fix: Introducing an AVL algorithm triggers height updates and tree rotations during insertions to enforce strict structural balance, ensuring that the tree height remains logarithmic relative to the data size.

Part (a): Design Verification & Metric Analysis

(i) Tree Shape and Expected Height for a Plain BST

- **Total Inserted Elements (n):** Over a peak hour with a sustained workload of 5,000 messages/second, the total number of items inserted is:
 $n = 5000 \text{ messages/sec} \times 3600 \text{ seconds/hour} = 18,000,000 \text{ messages}$
- **Tree Shape:** Because timestamps are strictly monotonic (always increasing) and the tree is keyed descending, each incoming receipt will consistently possess a timestamp greater than all previously inserted keys. In a descending key structure, larger values are routed to the left. Therefore, every new insert becomes the left child of the previous leaf node. This produces a completely **skewed tree (left-degenerate line)** resembling a single-linked list.
- **Expected Height:** A perfectly skewed tree with n elements achieves a worst-case height directly proportional to its element count.
 $\text{Expected Height} = n - 1 = 17,999,999 \text{ levels}$

(ii) Worst-Case 'Find Oldest Pending' Time on Plain BST (in Pointer Hops)

- **Operation Path:** The oldest pending receipt resides at the terminal end of the **leftmost descent** path.
- **Pointer Hops:** Because the tree has degraded into a straight left-skewed line of length n , reaching the deepest leftmost leaf from the root requires traversing every single node in the system sequentially.
 $\text{Worst-case Pointer Hops} = n - 1 = 17,999,999 \text{ hops}$

(iii) AVL Alternative Metric & SLA Implication Analysis

If the team upgrades the infrastructure to a self-balancing **AVL Tree alternative**, the height is strictly bounded by $1.44 \log_2(n)$.

- **AVL Tree Height (Worst-Case):**
 $\text{Height} \approx 1.44 \times \log_2(18,000,000) \approx 1.44 \times 24.1 \approx 34.7 \rightarrow 35 \text{ levels}$
- **Worst-Case Pointer Hops for AVL:** Finding the oldest receipt via leftmost descent will take at most the height of the tree.
 $\text{Worst-case Pointer Hops} = 35 \text{ hops}$

Result: Screenshots / Output

```
Listening on 50082
User program running
User program finished
--- Elasticsearch Multi-Term Query Engine ---
Loading initial posting lists...
list_machine : [1, 4, 7, 12, 15, 20, 24, 30]
list_learning : [2, 4, 8, 12, 18, 24, 30]
list_tutorial : [4, 6, 9, 12, 17, 24, 30, 35]

[Optimization] Order of intersection (shortest first):
List 1 (Size 7): [2, 4, 8, 12, 18, 24, 30]
List 2 (Size 8): [1, 4, 7, 12, 15, 20, 24, 30]
List 3 (Size 8): [4, 6, 9, 12, 17, 24, 30, 35]

Executing Intersection step 1...
-> Intermediate Matches: [4, 12, 24, 30]
Executing Intersection step 2...
-> Intermediate Matches: [4, 12, 24, 30]

-----
FINAL INTERSECTED MATCHES (Document IDs): [4, 12, 24, 30]
Time Complexity (worst case):  $O(|L1| + |L2| + |L3|)$ 
-----
Process finished with exit code 0
```

GitHub Link: https://github.com/kotamohith274/DSA-_2-Case-Studies.

Student Signature	Faculty Signature